

ML-ASSIGNMENT-2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
from sklearn.datasets import load_iris

import warnings
warnings.filterwarnings("ignore")

# Load dataset
df = pd.read_csv("train.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

✓ 6.4s

Dataset loaded successfully!
Shape: (1460, 81)

```
print("First 10 rows:")
display(df.head(10))
```

Python

First 10 rows:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	Pool
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	
5	6	50	RL	85.0	14115	Pave	NaN	IR1	Lvl	AllPub	...	
6	7	20	RL	75.0	10084	Pave	NaN	Reg	Lvl	AllPub	...	
7	8	60	RL	NaN	10382	Pave	NaN	IR1	Lvl	AllPub	...	
8	9	50	RM	51.0	6120	Pave	NaN	Reg	Lvl	AllPub	...	
9	10	190	RL	50.0	7420	Pave	NaN	Reg	Lvl	AllPub	...	

10 rows × 81 columns

```
rows, columns = df.shape

print("Number of rows:", rows)
print("Number of columns:", columns)
```

Python

Number of rows: 1460
Number of columns: 81

```
numerical_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include="object").columns

print("Numerical columns:")
print(list(numerical_columns))

print("\nNumber of numerical columns:", len(numerical_columns))

print("\nCategorical columns:")
print(list(categorical_columns))

print("\nNumber of categorical columns:", len(categorical_columns))
```

Python

Numerical columns:
['Id', 'MSSubClass', 'LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
Number of numerical columns: 38
Categorical columns:
['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neighbo
Number of categorical columns: 43

```
print("Data types of all columns:")
print(df.dtypes)
```

Python

Data types of all columns:
Id int64
MSSubClass int64
MSZoning object
LotFrontage float64
LotArea int64
...
MoSold int64
YrSold int64
SaleType object
SaleCondition object
SalePrice int64
Length: 81, dtype: object

```
print("\nColumns with missing/unknown datatype:")
print(df.dtypes[df.dtypes == "object"])

print("\nDataset information:")
df.info()
```

Py

Columns with missing/unknown datatype:
MSZoning object
Street object
Alley object
LotShape object
LandContour object
Utilities object
LotConfig object
LandSlope object
Neighborhood object

```

Condition2      object
BldgType        object
HouseStyle      object
RoofStyle       object
RoofMat1        object
Exterior1st     object
Exterior2nd     object
MasVnrType      object
ExterQual       object
ExterCond       object
Foundation      object
BsmtQual        object
BsmtCond        object
...
79 SaleCondition 1460 non-null object
80 SalePrice     1460 non-null int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```

features = [
    "LotArea",
    "GrLivArea",
    "SalePrice",
    "YearBuilt",
    "OverallQual"
]

plt.figure(figsize=(15, 10))

for i, feature in enumerate(features):
    plt.subplot(3, 2, i + 1)

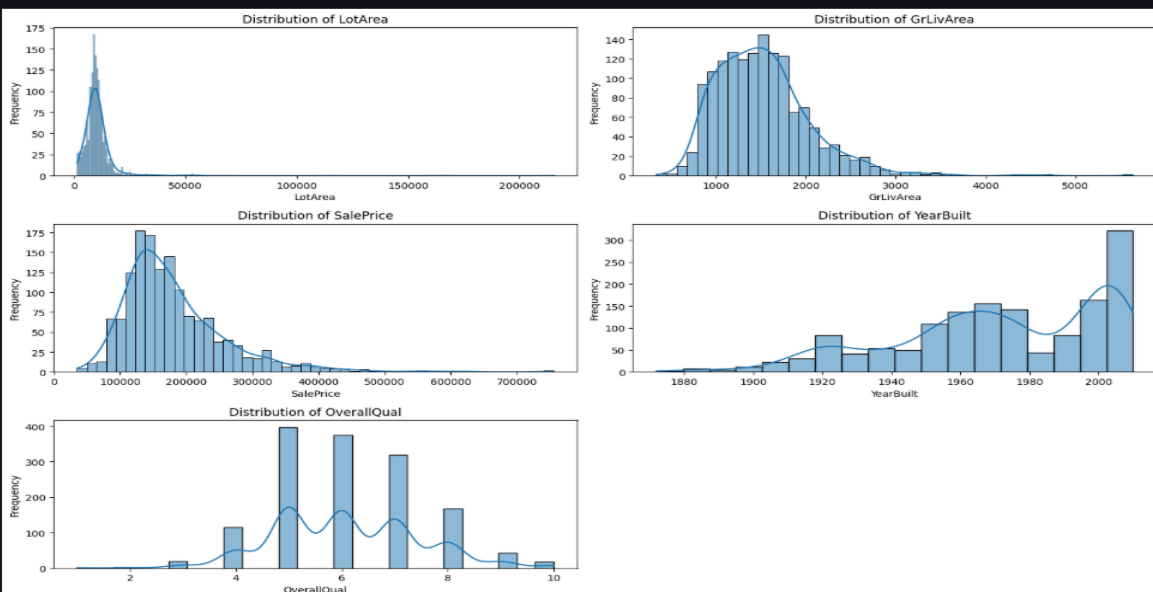
```

```

plt.xlabel(feature)
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

```



```
print("Skewness of selected numerical features:\n")
```

```
for feature in features:  
    skewness = df[feature].skew()  
    print(feature, ":", round(skewness, 3))
```

Skewness of selected numerical features:

LotArea : 12.208
GrLivArea : 1.367
SalePrice : 1.883
YearBuilt : -0.613
OverallQual : 0.217

```
print("""  
Interpretation:
```

```
Skewness close to 0 -> approximately symmetric/normal  
Positive skewness   -> right-skewed  
Negative skewness   -> left-skewed
```

```
LotArea, GrLivArea and SalePrice are generally right-skewed.  
OverallQual is closer to a normal/symmetric distribution.  
YearBuilt is comparatively less skewed.  
""")
```

Interpretation:

```
Skewness close to 0 -> approximately symmetric/normal  
Positive skewness   -> right-skewed  
Negative skewness   -> left-skewed
```

LotArea, GrLivArea and SalePrice are generally right-skewed.
OverallQual is closer to a normal/symmetric distribution.
YearBuilt is comparatively less skewed.

```
categorical_features = [  
    "Neighborhood",  
    "HouseStyle",  
    "BldgType"  
]
```

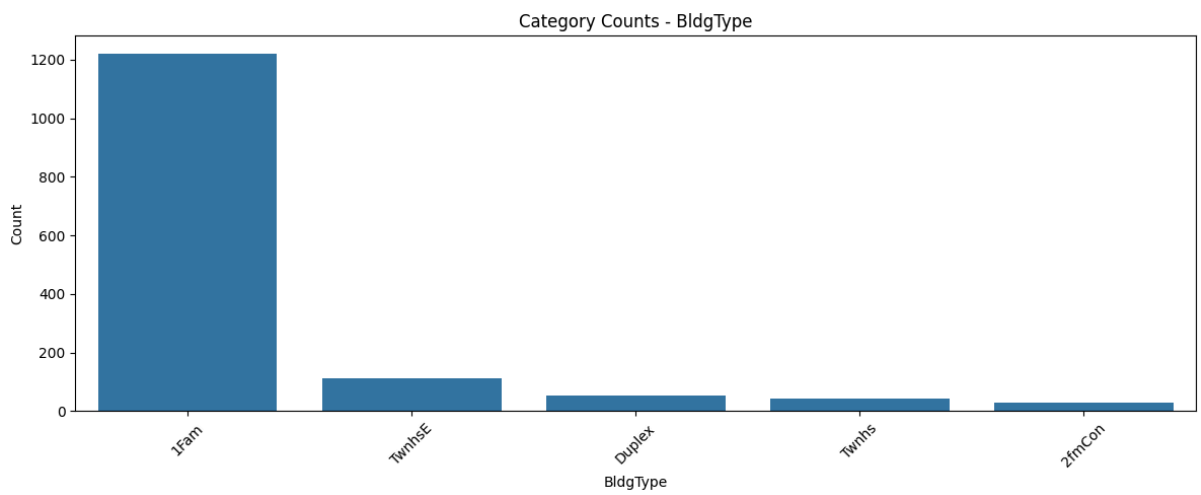
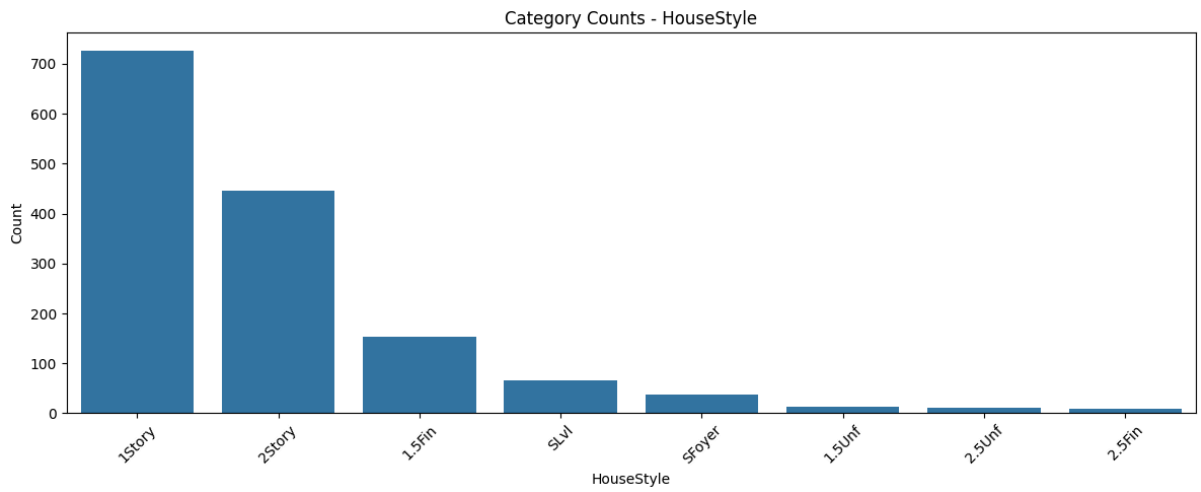
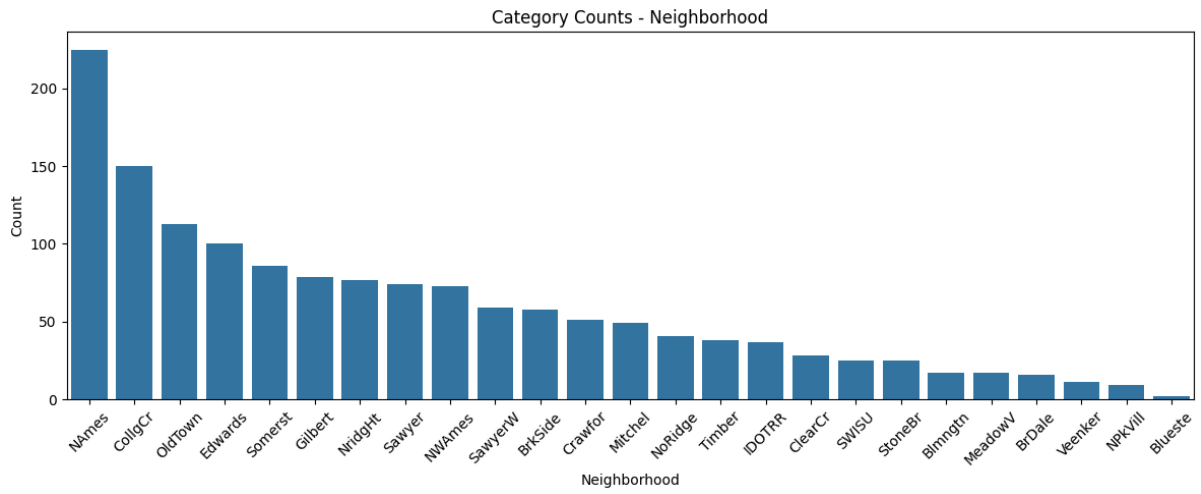
```
for feature in categorical_features:
```

```
    plt.figure(figsize=(12, 5))
```

```
    counts = df[feature].value_counts()
```

```
    sns.barplot(  
        x=counts.index,  
        y=counts.values  
    )
```

```
    plt.title("Category Counts - " + feature)  
    plt.xlabel(feature)  
    plt.ylabel("Count")  
    plt.xticks(rotation=45)
```



```
for feature in categorical_features:

    counts = df[feature].value_counts()

    print("\n", feature)

    print("Most common category:")
    print(counts.idxmax(), "->", counts.max())

    print("Rarest category:")
    print(counts.idxmin(), "->", counts.min())
```

```
Neighborhood
Most common category:
NAmes -> 225
Rarest category:
Blueste -> 2
```

```
HouseStyle
Most common category:
1Story -> 726
Rarest category:
2.5Fin -> 8
```

```
BldgType
Most common category:
1Fam -> 1220
Rarest category:
2fmCon -> 31
```

```
for feature in categorical_columns:
    print("\nTop 10 categories in", feature)
    print(df[feature].value_counts().head(10))
```

[19]

...

Top 10 categories in MSZoning

MSZoning

RL 1151

RM 218

FV 65

RH 16

C (all) 10

Name: count, dtype: int64

Top 10 categories in Street

Street

Pave 1454

Grvl 6

Name: count, dtype: int64

Top 10 categories in Alley

Alley

Grvl 50

Pave 41

Name: count, dtype: int64

Top 10 categories in LotShape

LotShape

Reg 925

...

Family 20

```
Family      20
Alloca      12
AdjLand      4
Name: count, dtype: int64
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output

```
for feature in categorical_columns:

    counts = df[feature].value_counts()

    if len(counts) > 0:

        dominant_percentage = (
            counts.iloc[0] / len(df)
        ) * 100

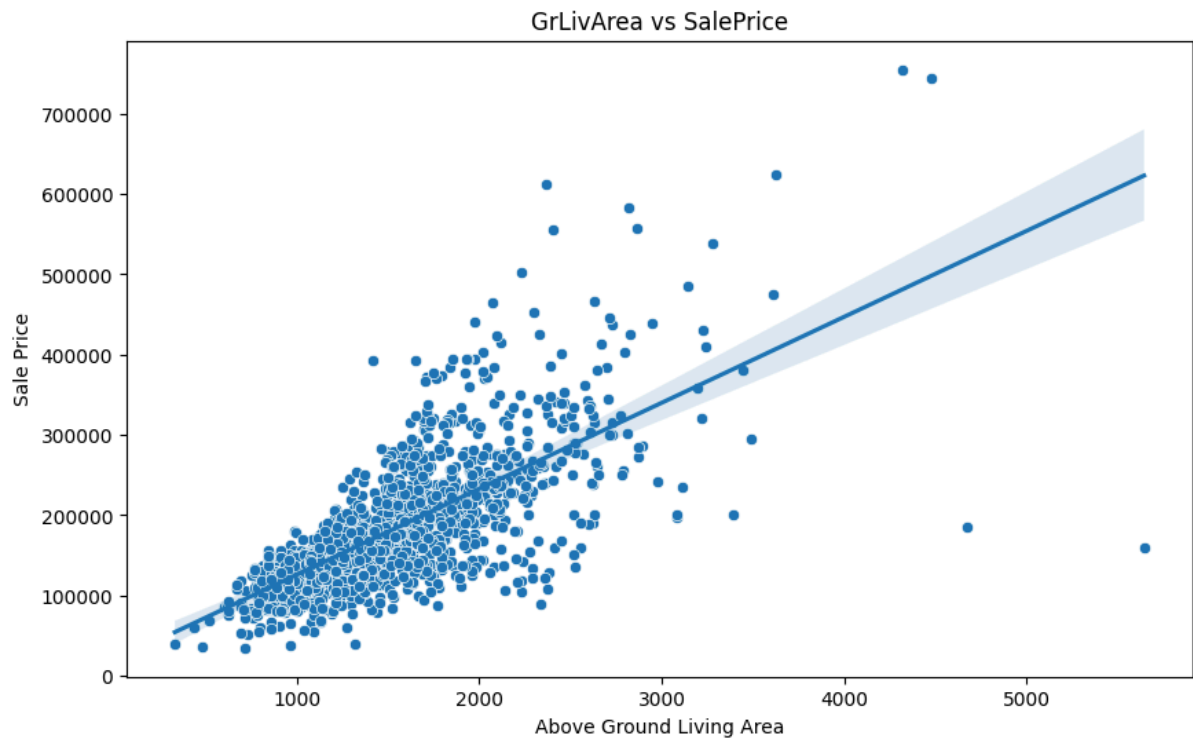
        if dominant_percentage > 50:
            print(
                feature,
                "-> Dominant category:",
                counts.index[0],
                "(",
                round(dominant_percentage, 2),
                "%)"
            )
```

```
MSZoning -> Dominant category: RL ( 78.84 %)
Street -> Dominant category: Pave ( 99.59 %)
LotShape -> Dominant category: Reg ( 63.36 %)
LandContour -> Dominant category: Lvl ( 89.79 %)
Utilities -> Dominant category: AllPub ( 99.93 %)
LotConfig -> Dominant category: Inside ( 72.05 %)
```

```
LandSlope -> Dominant category: Gtl ( 94.66 %)
Condition1 -> Dominant category: Norm ( 86.3 %)
Condition2 -> Dominant category: Norm ( 98.97 %)
BldgType -> Dominant category: 1Fam ( 83.56 %)
RoofStyle -> Dominant category: Gable ( 78.15 %)
RoofMatl -> Dominant category: CompShg ( 98.22 %)
ExterQual -> Dominant category: TA ( 62.05 %)
ExterCond -> Dominant category: TA ( 87.81 %)
BsmtCond -> Dominant category: TA ( 89.79 %)
BsmtExposure -> Dominant category: No ( 65.27 %)
BsmtFinType2 -> Dominant category: Unf ( 86.03 %)
Heating -> Dominant category: GasA ( 97.81 %)
HeatingQC -> Dominant category: Ex ( 50.75 %)
CentralAir -> Dominant category: Y ( 93.49 %)
Electrical -> Dominant category: SBrkr ( 91.37 %)
KitchenQual -> Dominant category: TA ( 50.34 %)
Functional -> Dominant category: Typ ( 93.15 %)
GarageType -> Dominant category: Attchd ( 59.59 %)
GarageQual -> Dominant category: TA ( 89.79 %)
GarageCond -> Dominant category: TA ( 90.82 %)
PavedDrive -> Dominant category: Y ( 91.78 %)
SaleType -> Dominant category: WD ( 86.78 %)
SaleCondition -> Dominant category: Normal ( 82.05 %)
```

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="GrLivArea",
    y="SalePrice"
)
```



There is a positive relationship between GrLivArea and SalePrice. Generally, as the above-ground living area increases, the sale price also tends to increase.

However, some outliers are present.
""")

Observation:

There is a positive relationship between GrLivArea and SalePrice. Generally, as the above-ground living area increases, the sale price also tends to increase.

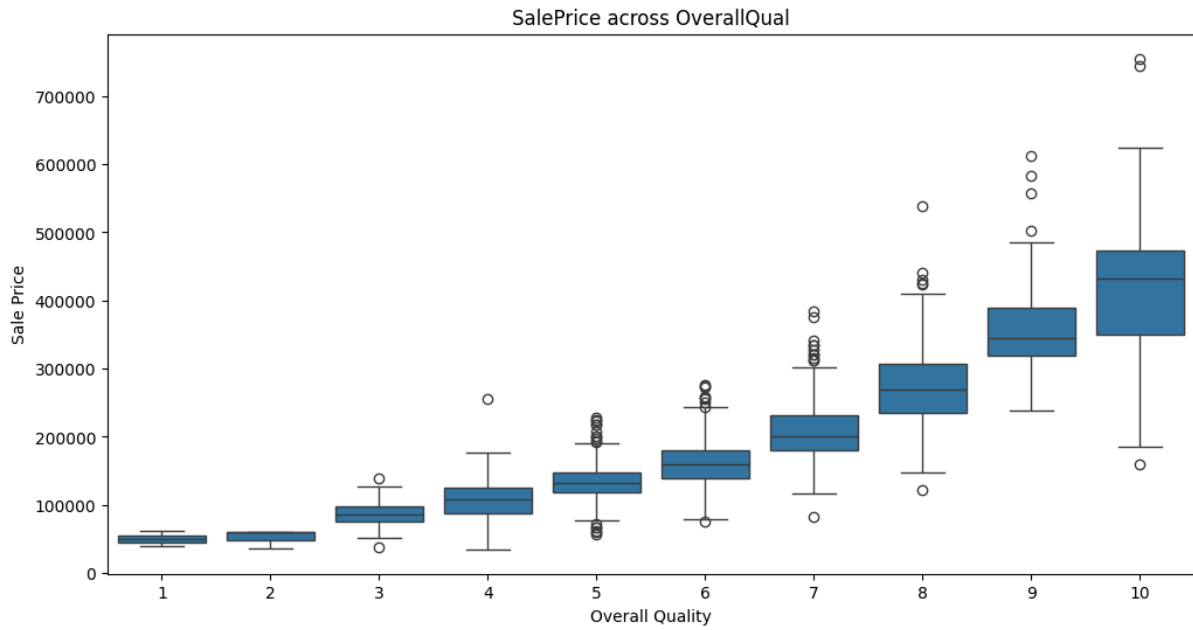
However, some outliers are present.

```
plt.figure(figsize=(12, 6))

sns.boxplot(
    data=df,
    x="OverallQual",
    y="SalePrice"
)

plt.title("SalePrice across OverallQual")
plt.xlabel("Overall Quality")
plt.ylabel("Sale Price")

plt.show()
```



```
print("Top 10 features correlated with SalePrice:")
print(saleprice_corr.head(11))
```

Top 10 features correlated with SalePrice:

SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
YearRemodAdd	0.507101

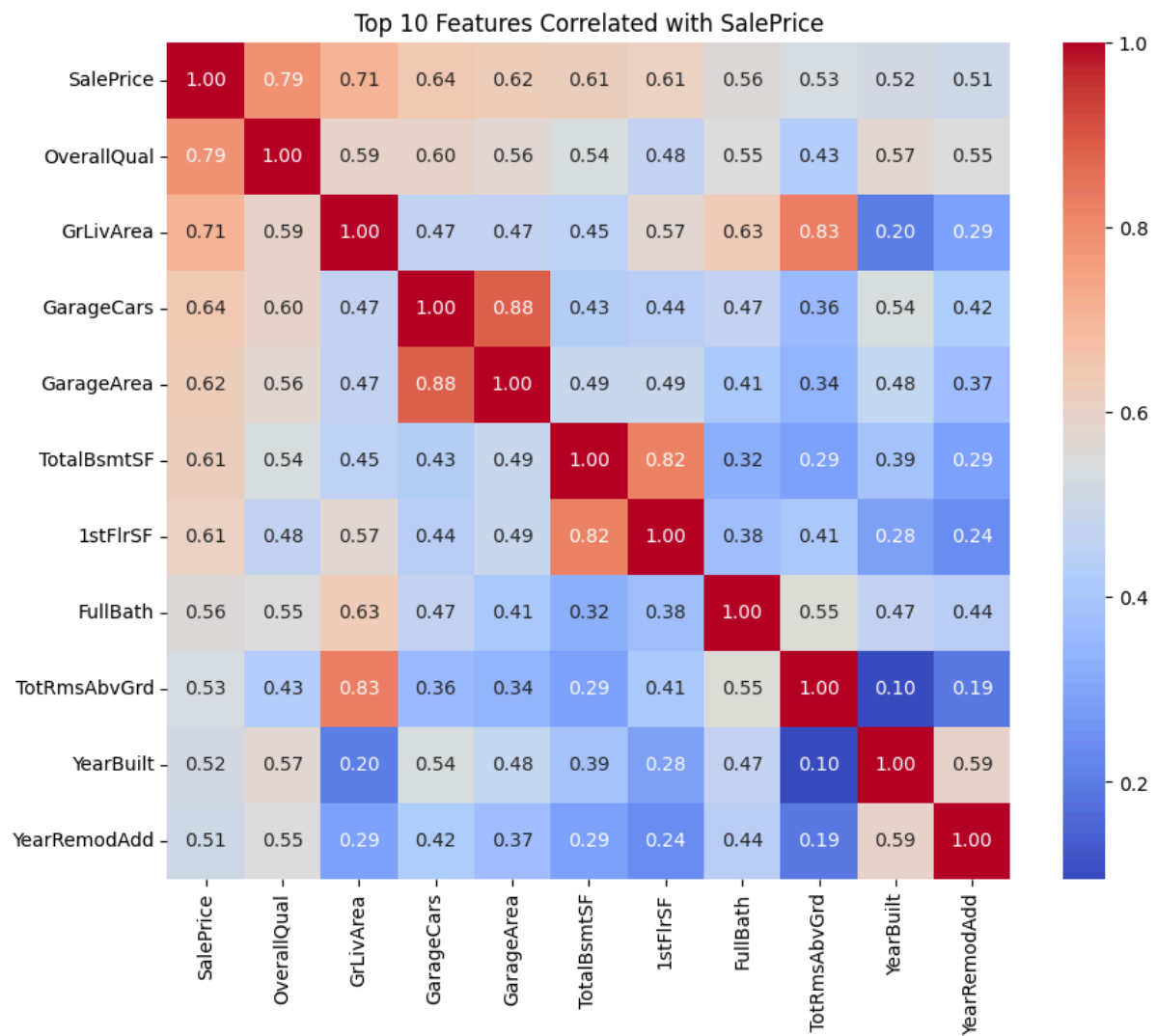
Name: SalePrice, dtype: float64

```
plt.figure(figsize=(10, 8))

sns.heatmap(
    df[top_10_features].corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Top 10 Features Correlated with SalePrice")

plt.show()
```



```
neighborhood_price = (
    df.groupby("Neighborhood")["SalePrice"]
      .mean()
      .sort_values(ascending=False)
)

print("Average SalePrice by Neighborhood:")
print(neighborhood_price)
```

```
Average SalePrice by Neighborhood:
Neighborhood
NoRidge      335295.317073
NridgHt      316270.623377
StoneBr      310499.000000
Timber       242247.447368
Veenker      238772.727273
Somerst      225379.837209
ClearCr      212565.428571
Crawfor      210624.725490
CollgCr      197965.773333
Blmngtn      194870.882353
Gilbert      192854.506329
NWAmes       189050.068493
SawyerW      186555.796610
Mitchel      156270.122449
NAmes        145847.080000
NPKvill      142694.444444
SWISU        142591.360000
Blueste      137500.000000
Sawyer       136793.135135
OldTown      128225.300885
Edwards      128219.700000
```

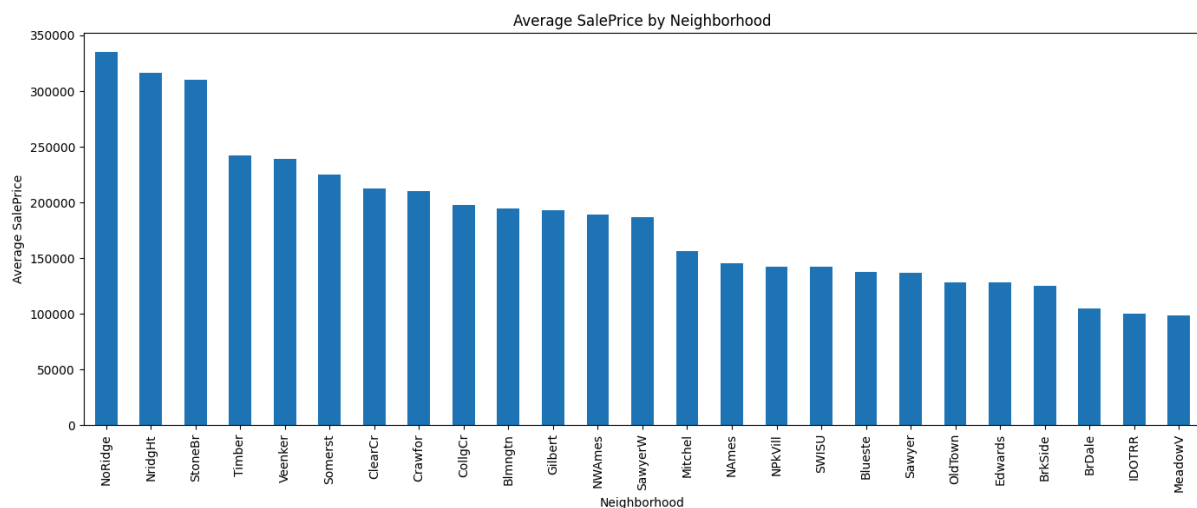
```
plt.figure(figsize=(14, 6))

neighborhood_price.plot(kind="bar")

plt.title("Average SalePrice by Neighborhood")
plt.xlabel("Neighborhood")
plt.ylabel("Average SalePrice")

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```



```
print("Top 2 neighborhoods by average SalePrice:")
print(neighborhood_price.head(2))
```

```
Top 2 neighborhoods by average SalePrice:
Neighborhood
NoRidge      335295.317073
NridgHt      316270.623377
Name: SalePrice, dtype: float64
```

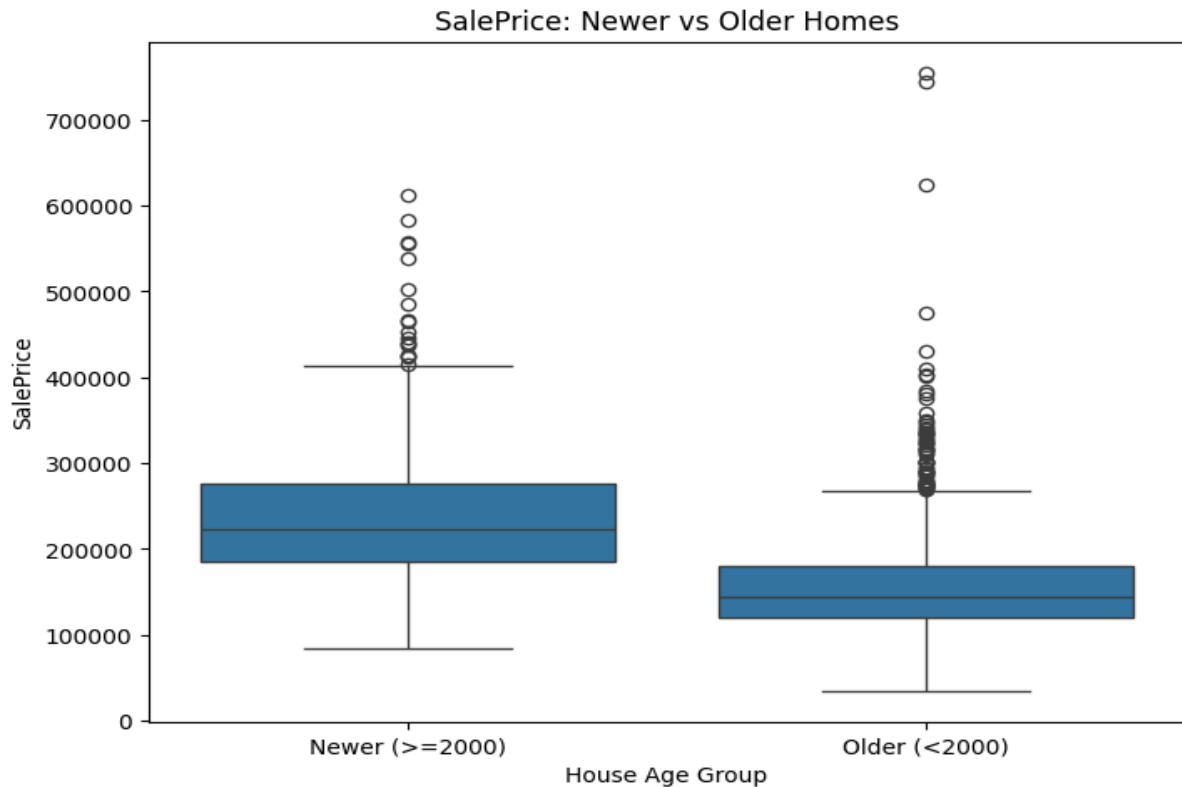
```
df["HouseAgeGroup"] = np.where(
    df["YearBuilt"] >= 2000,
    "Newer (>=2000)",
    "Older (<2000)"
)

print(df["HouseAgeGroup"].value_counts())
```

```
HouseAgeGroup
Older (<2000)      1072
Newer (>=2000)      388
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 6))

sns.boxplot(
    data=df,
    x="HouseAgeGroup",
    y="SalePrice"
```



```
newer_prices = df[
    df["YearBuilt"] >= 2000
]["SalePrice"].dropna()

older_prices = df[
    df["YearBuilt"] < 2000
]["SalePrice"].dropna()

t_statistic, p_value = ttest_ind(
    newer_prices,
    older_prices,
    equal_var=False
)

print("T-statistic:", t_statistic)
print("P-value:", p_value)

statistic: 18.001293991506017
value: 1.0987021443774397e-57

if p_value < 0.05:
    print("""
Conclusion:
Since p-value < 0.05, we reject the null hypothesis.

There is a statistically significant difference
in mean SalePrice between newer and older homes.
""")
else:
    print("""
```

```

missing_values = df.isnull().sum()

missing_values = missing_values[
    missing_values > 0
].sort_values(ascending=False)

print("Columns containing missing values:")
print(missing_values)

```

[33]

... Columns containing missing values:

PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872
FireplaceQu	690
LotFrontage	259
GarageType	81
GarageYrBlt	81
GarageFinish	81
GarageQual	81
GarageCond	81
BsmtExposure	38
BsmtFinType2	38
BsmtQual	37
BsmtCond	37
BsmtFinType1	37
MasVnrArea	8
Electrical	1
dtype: int64	

```
top_5_missing = df.isnull().sum().sort_values(  
    ascending=False  
)<div data-bbox="136 205 421 221" data-label="Text">

Top 5 columns with missing values:


```

PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872

dtype: int64

```
df["LotFrontage"] = df.groupby(  
    "Neighborhood"  
)["LotFrontage"].transform(  
    lambda x: x.fillna(x.median())  
)
```

✓ 0.0s

```
df["LotFrontage"] = df["LotFrontage"].fillna(  
    df["LotFrontage"].median()  
)
```

✓ 0.0s

Generate

+ Code

+ Markdown

```
numerical_columns = df.select_dtypes(  
    include=np.number  
)<div>).columns
```

```
for column in numerical_columns:
```

```
    df[column] = df[column].fillna(  
        df[column].median()  
    )
```

✓ 0.0s

```
categorical_columns = df.select_dtypes(  
    include="object"  
)<div>).columns
```

```
for column in categorical_columns:
```

```
    if df[column].isnull().sum() > 0:
```

```
        mode_value = df[column].mode()
```

```
        if len(mode_value) > 0:
```

```
            df[column] = df[column].fillna(  
                mode_value[0]  
            )
```

```
        else:
```

```
            df[column] = df[column].fillna(  
                "Missing"  
            )
```

```
remaining_missing = df.isnull().sum().sum()

print("Total remaining missing values:", remaining_missing)

if remaining_missing == 0:
    print("All missing values have been successfully handled.")
else:
    print("Some missing values are still present.")
```

Total remaining missing values: 0
All missing values have been successfully handled.

```
def detect_outliers_iqr(data, column):

    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = data[
        (data[column] < lower_bound) |
        (data[column] > upper_bound)
    ]

    return outliers, lower_bound, upper_bound
```

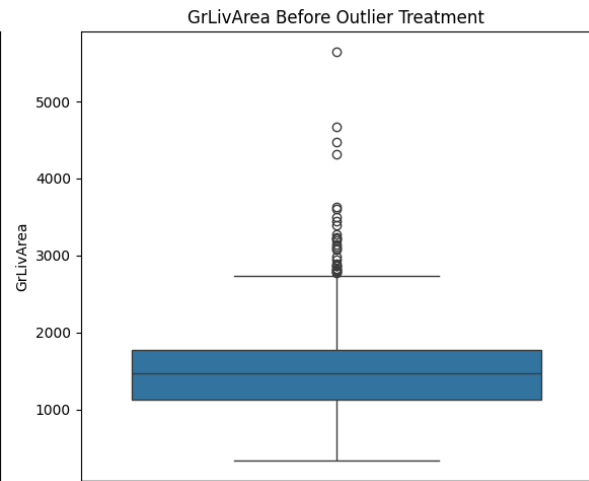
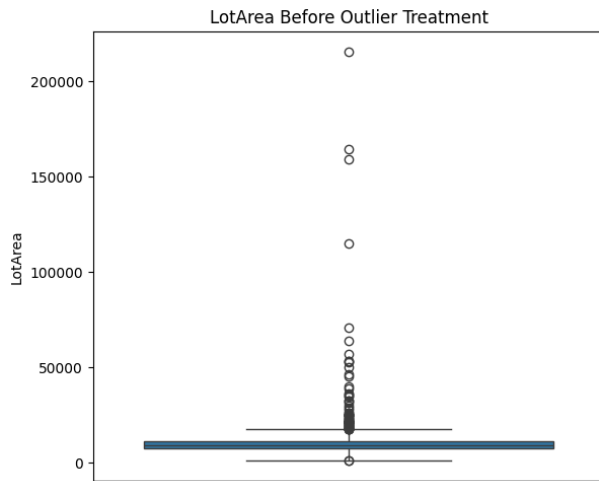
```
lot_outliers, lot_lower, lot_upper = detect_outliers_iqr(  
    df,  
    "LotArea"  
)  
  
print("LotArea outliers:", len(lot_outliers))  
  
print("Lower bound:", lot_lower)  
print("Upper bound:", lot_upper)
```

```
LotArea outliers: 69  
Lower bound: 1481.5  
Upper bound: 17673.5
```

```
grliv_outliers, grliv_lower, grliv_upper = detect_outliers_iqr(  
    df,  
    "GrLivArea"  
)  
  
print("GrLivArea outliers:", len(grliv_outliers))  
  
print("Lower bound:", grliv_lower)  
print("Upper bound:", grliv_upper)
```

```
GrLivArea outliers: 31  
Lower bound: 158.625  
Upper bound: 2747.625
```

```
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
  
sns.boxplot(y=df["LotArea"])  
  
plt.title("LotArea Before Outlier Treatment")  
  
plt.subplot(1, 2, 2)  
  
sns.boxplot(y=df["GrLivArea"])  
  
plt.title("GrLivArea Before Outlier Treatment")  
  
plt.tight_layout()  
plt.show()
```



```
def cap_outliers_iqr(data, column):

    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    data[column] = data[column].clip(
        lower=lower_bound,
        upper=upper_bound
    )
```

0.0s

```
cap_outliers_iqr(df, "LotArea")
cap_outliers_iqr(df, "GrLivArea")

print("Outlier capping completed.")
```

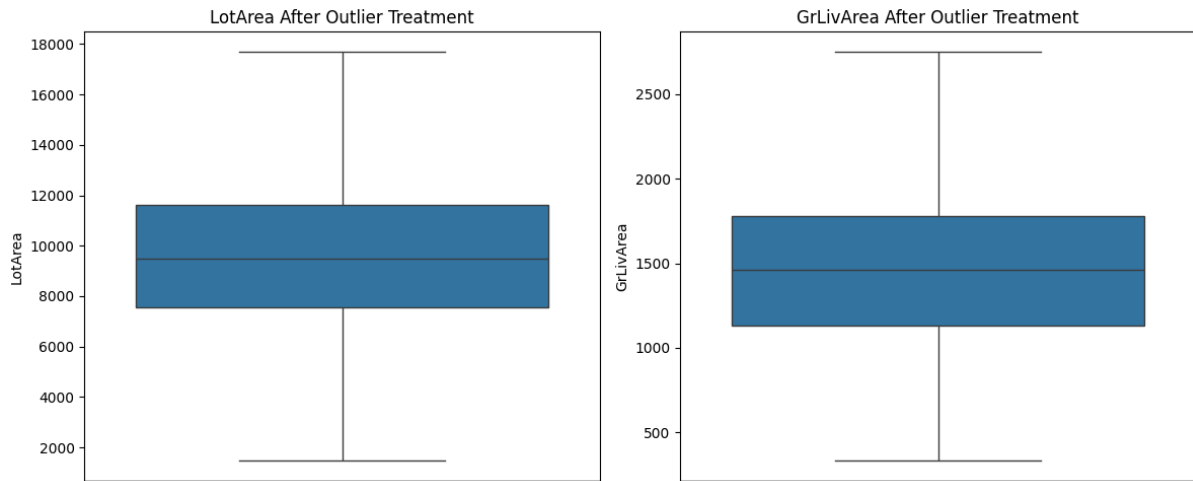
0.0s

Outlier capping completed.

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

sns.boxplot(y=df["LotArea"])
```



```
plt.figure(figsize=(10, 6))

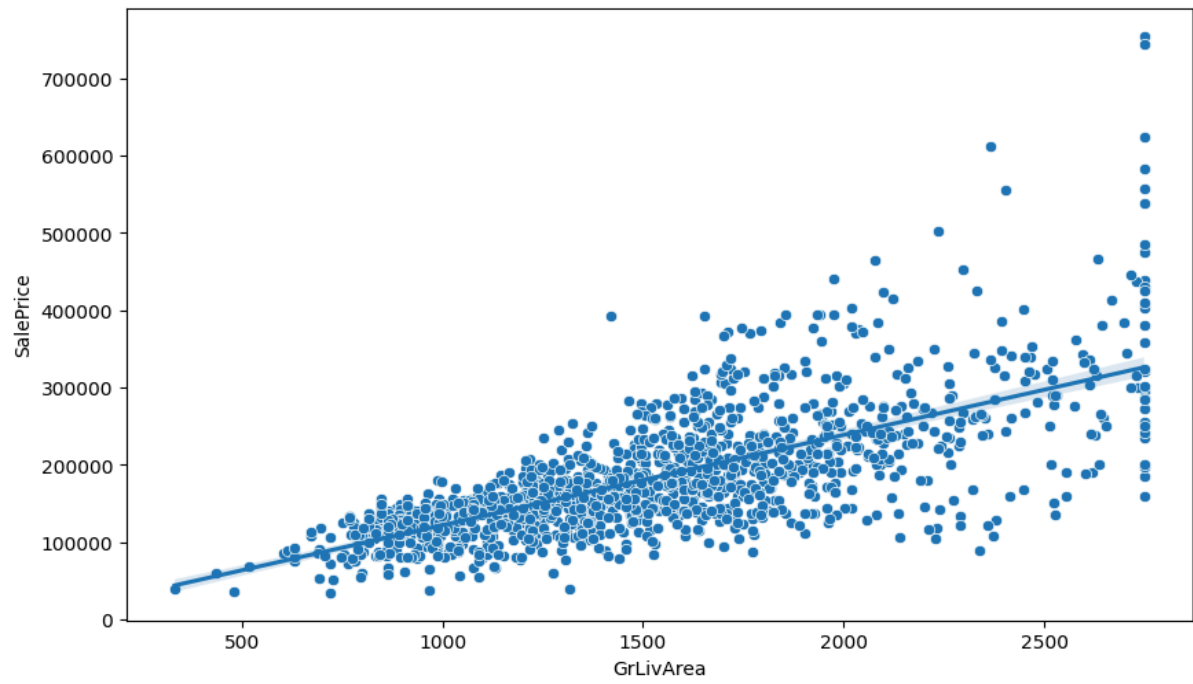
sns.scatterplot(
    data=df,
    x="GrLivArea",
    y="SalePrice"
)

sns.regplot(
    data=df,
    x="GrLivArea",
    y="SalePrice",
    scatter=False
)

plt.title("GrLivArea vs SalePrice After Outlier Treatment")
plt.xlabel("GrLivArea")
plt.ylabel("SalePrice")

plt.show()
```

GrLivArea vs SalePrice After Outlier Treatment



```
# Select numerical columns
numerical_features = df.select_dtypes(
    include=np.number
).columns

# Remove ID because it is an identifier, not a meaningful feature
numerical_features = numerical_features.drop(
    "Id",
    errors="ignore"
)

scaler = StandardScaler()

df_standardized = df.copy()

df_standardized[numerical_features] = scaler.fit_transform(
    df_standardized[numerical_features]
)

print("StandardScaler applied successfully.")
```

✓ 0.0s

StandardScaler applied successfully.

```
comparison_features = [
    "GrLivArea",
    "LotArea",
    "OverallQual",
    "SalePrice"
]
```

```
print("\nAfter Standard Scaling:")
display(df_standardized[comparison_features].describe().loc[
|     ["mean", "std", "min", "max"]
| ])
✓ 0.0s
```

Before scaling:

	GrLivArea	LotArea	OverallQual	SalePrice
mean	1503.735873	9647.388014	6.099315	180921.195890
std	481.375641	3594.356399	1.382997	79442.502883
min	334.000000	1481.500000	1.000000	34900.000000
max	2747.625000	17673.500000	10.000000	755000.000000

After Standard Scaling:

	GrLivArea	LotArea	OverallQual	SalePrice
mean	3.771717e-17	5.535907e-17	1.387018e-16	1.362685e-16
std	1.000343e+00	1.000343e+00	1.000343e+00	1.000343e+00
min	-2.430818e+00	-2.272642e+00	-3.688413e+00	-1.838704e+00
max	2.584915e+00	2.233741e+00	2.821425e+00	7.228819e+00

```

minmax_scaler = MinMaxScaler()

df_minmax = df.copy()

df_minmax[numerical_features] = minmax_scaler.fit_transform(
    df_minmax[numerical_features]
)

print("MinMaxScaler applied successfully.")

```

✓ 0.0s

inMaxScaler applied successfully.

```

output_file = "house_prices_clean.csv"

df.to_csv(
    output_file,
    index=False
)

print("Cleaned dataset saved as:", output_file)

```

✓ 0.0s

leaned dataset saved as: house_prices_clean.csv

 Generate

 Code

 Markdown

```

iris = load_iris()

X = iris.data
y = iris.target

```

```

print("Feature names:")
print(iris.feature_names)

print("\nTarget names:")
print(iris.target_names)

print("\nDataset shape:", X.shape)

```

✓ 0.0s

Feature names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target names:
['setosa' 'versicolor' 'virginica']

Dataset shape: (150, 4)

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

```

✓ 0.0s

Training samples: 120

Testing samples: 30

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed.")
```

✓ 0.0s

Feature scaling completed.

```
knn_euclidean = KNeighborsClassifier(
    n_neighbors=5,
    metric="euclidean"
)

knn_euclidean.fit(
    X_train_scaled,
    y_train
)

y_pred_euclidean = knn_euclidean.predict(
    X_test_scaled
)
```

✓ 0.0s

```
accuracy = accuracy_score(
    y_test,
    y_pred_euclidean
)

precision = precision_score(
    y_test,
    y_pred_euclidean,
    average="weighted"
)

recall = recall_score(
    y_test,
    y_pred_euclidean,
    average="weighted"
)

f1 = f1_score(
    y_test,
    y_pred_euclidean,
    average="weighted"
)

print("Euclidean KNN")
print("-----")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

✓ 0.0s

Euclidean KNN

Accuracy : 0.9333333333333333
Precision: 0.9444444444444445
Recall : 0.9333333333333333
F1 Score : 0.9326599326599326

```
print(  
    classification_report(  
        y_test,  
        y_pred_euclidean,  
        target_names=iris.target_names  
    )  
)
```

✓ 0.0s

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.83	1.00	0.91	10
virginica	1.00	0.80	0.89	10
accuracy			0.93	30
macro avg	0.94	0.93	0.93	30
weighted avg	0.94	0.93	0.93	30

Generate

+ Code

+ Markdown

Add Markdown Cell

```
knn_manhattan = KNeighborsClassifier(  
    n_neighbors=5,  
    metric="manhattan"
```

```
manhattan_accuracy = accuracy_score(  
    y_test,  
    y_pred_manhattan  
)  
print("Manhattan KNN Accuracy:",  
      manhattan_accuracy)
```

✓ 0.0s

Manhattan KNN Accuracy: 0.9333333333333333

```
k_values = [  
    1,  
    3,  
    5,  
    7,  
    9,  
    11,  
    15,  
    21  
]  
  
euclidean_accuracies = []  
manhattan_accuracies = []  
  
for k in k_values:  
    # Euclidean  
    model_e = KNeighborsClassifier(  
        n_neighbors=k,  
        metric="euclidean"  
    )
```

```

results = pd.DataFrame({
    "K": k_values,
    "Euclidean Accuracy": euclidean_accuracies,
    "Manhattan Accuracy": manhattan_accuracies
})

display(results)

```

✓ 0.1s

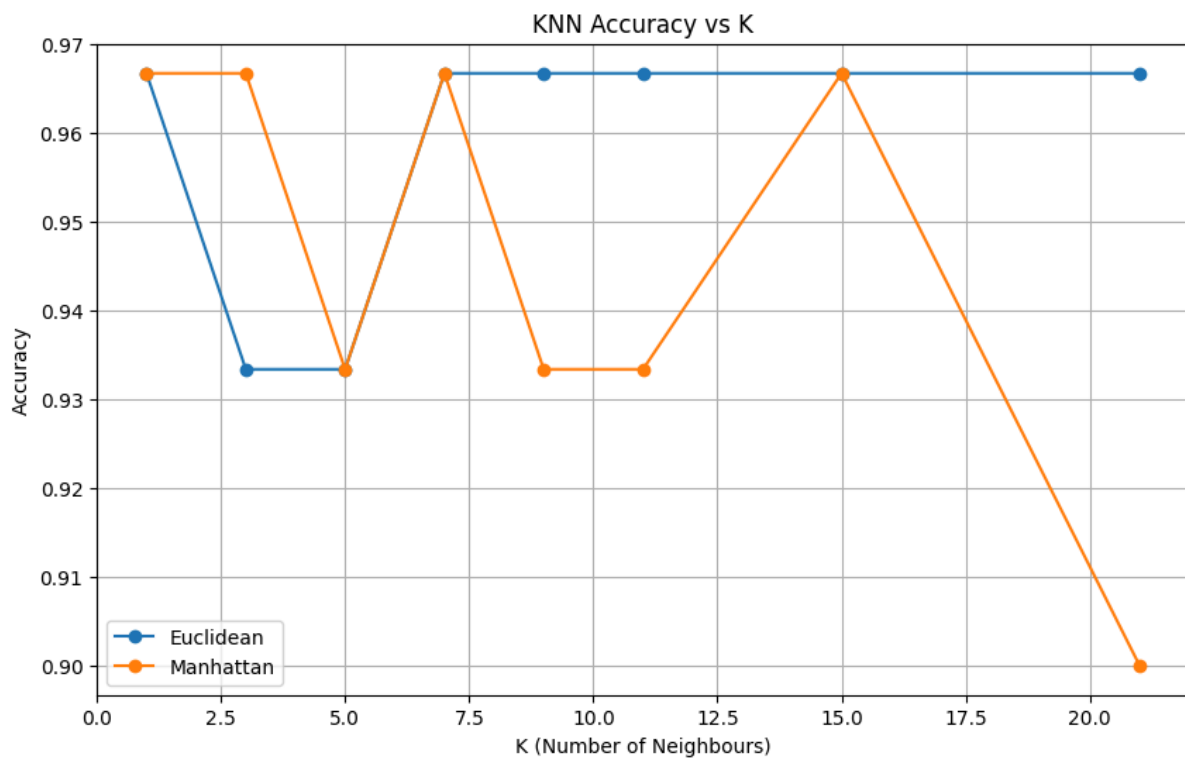
	K	Euclidean Accuracy	Manhattan Accuracy
0	1	0.966667	0.966667
1	3	0.933333	0.966667
2	5	0.933333	0.933333
3	7	0.966667	0.966667
4	9	0.966667	0.933333
5	11	0.966667	0.933333
6	15	0.966667	0.966667
7	21	0.966667	0.900000

```

plt.figure(figsize=(10, 6))

plt.plot(
    k_values,
    euclidean_accuracies,
    marker="o",
    label="Euclidean"
)

```



```
best_euclidean_index = np.argmax(  
    euclidean_accuracies  
)  
  
best_manhattan_index = np.argmax(  
    manhattan_accuracies  
)  
  
best_euclidean_k = k_values[  
    best_euclidean_index  
)  
  
best_manhattan_k = k_values[  
    best_manhattan_index  
)  
  
best_euclidean_accuracy = euclidean_accuracies[  
    best_euclidean_index  
)  
  
best_manhattan_accuracy = manhattan_accuracies[  
    best_manhattan_index  
)  
  
print(  
    "Best Euclidean K:",  
    best_euclidean_k  
)  
  
print(  
    "Best Euclidean Accuracy:",  
    best_euclidean_accuracy  
)
```

```
print(
    "\nBest Manhattan K:",
    best_manhattan_k
)

print(
    "Best Manhattan Accuracy:",
    best_manhattan_accuracy
)
```

✓ 0.0s

Best Euclidean K: 1
Best Euclidean Accuracy: 0.9666666666666667

Best Manhattan K: 1
Best Manhattan Accuracy: 0.9666666666666667

```
if best_euclidean_accuracy >= best_manhattan_accuracy:

    best_k = best_euclidean_k
    best_metric = "euclidean"

else:

    best_k = best_manhattan_k
    best_metric = "manhattan"

final_knn = KNeighborsClassifier(
    n_neighbors=best_k,
    metric=best_metric
```

```

final_knn.fit(
    X_train_scaled,
    y_train
)

final_prediction = final_knn.predict(
    X_test_scaled
)

print("Best K:", best_k)
print("Best Distance Metric:", best_metric)
print(
    "Final Accuracy:",
    accuracy_score(
        y_test,
        final_prediction
    )
)

```

✓ 0.0s

```

Best K: 1
Best Distance Metric: euclidean
Final Accuracy: 0.9666666666666667

```

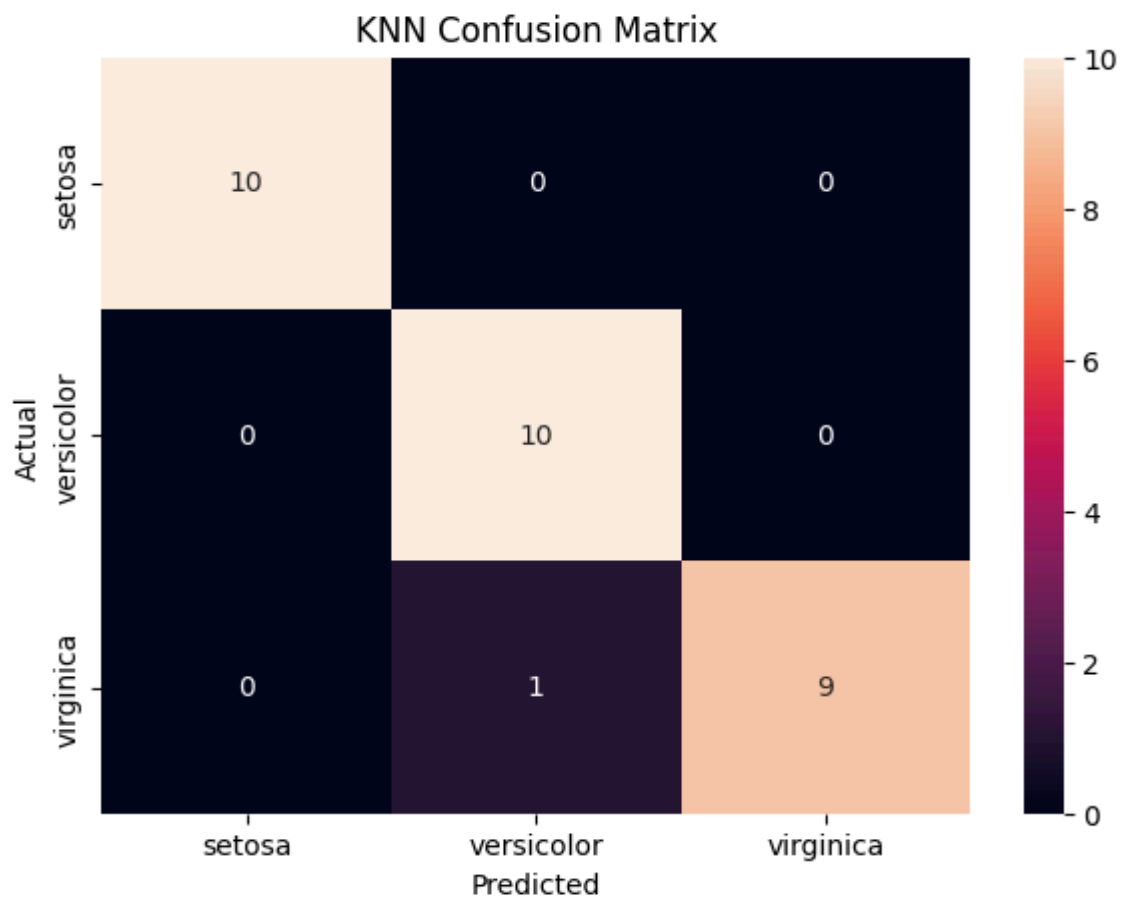
```

print(
    classification_report(
        y_test,
        final_prediction,
        target_names=iris.target_names
    )
)

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.91	1.00	0.95	10
virginica	1.00	0.90	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

```
cm = confusion_matrix(  
    y_test,  
    final_prediction  
)  
  
plt.figure(figsize=(7, 5))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    xticklabels=iris.target_names,  
    yticklabels=iris.target_names  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("KNN Confusion Matrix")  
  
plt.show()
```



===== FINAL CONCLUSION =====

PART A:

The House Prices dataset was successfully loaded and inspected.

PART B:

Numerical and categorical features were analyzed using histograms and bar charts. Skewness was calculated.

PART C:

Bivariate relationships were analyzed using scatter plots, boxplots, correlation analysis and a t-test.

PART D:

Missing values were handled using:

- Median for numerical features
- Mode/"Missing" for categorical features
- Neighborhood-wise median for LotFrontage

PART E:

Outliers in LotArea and GrLivArea were detected using the IQR method and capped at the IQR boundaries.

PART F:

...

The best K and distance metric were selected based on classification accuracy.